

COMPREHENSIVE REPORT

Generative AI & Large Language Models

Subject	19DT608 — Prompt Engineering
Name	AMIRDAVARSHINI D
Register No.	212225230013
Slot	T2-H16
Target Audience	Students & Technical Professionals
Report Type	Comprehensive Academic Report

TABLE OF CONTENTS
1. Foundational Concepts of Generative AI
2. Generative AI Architectures (Transformers & More)
3. Generative AI Architecture & Its Applications
4. Impact of Scaling in LLMs
5. Large Language Models — How They Are Built
6. Limitations, Ethics & Future Trends

1

Foundational Concepts of Generative AI

1.1 What is Artificial Intelligence?

Artificial Intelligence (AI) refers to the simulation of human intelligence in machines programmed to think, learn, and make decisions. It encompasses a broad set of techniques that allow computers to perform tasks that traditionally required human reasoning — from recognizing speech to making strategic decisions.

1.2 Machine Learning — The Engine Behind AI

Machine Learning (ML) is a subset of AI that enables systems to automatically learn and improve from experience without being explicitly programmed for every scenario. Instead of following hard-coded rules, ML models identify patterns in data and generalize to new situations.

Type	How It Learns	Example Use Case
Supervised Learning	From labeled input-output pairs	Email spam detection
Unsupervised Learning	Finding patterns in unlabeled data	Customer segmentation
Reinforcement Learning	Rewards and penalties from environment	Game-playing AI (AlphaGo)

1.3 Generative AI — Creating, Not Just Classifying

Generative AI is a revolutionary branch of AI focused on **creating new content**. While traditional discriminative AI asks 'What category does this belong to?', Generative AI asks 'What new data can I create that resembles what I've learned?' It models the underlying probability distribution of data and samples from it to generate novel outputs.

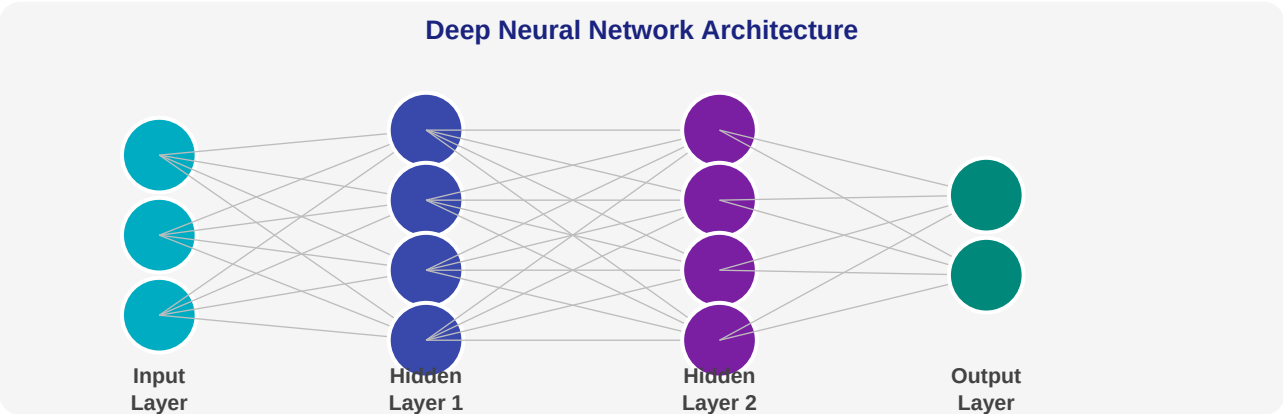


Figure 1: Deep Neural Network — the foundation of all modern AI models

Key Characteristics of Generative AI:

- **Learns data distributions** — understands the statistical patterns underlying training data
- **Produces realistic new content** — text, images, audio, video, and code
- **Uses deep learning** — specifically transformer-based architectures
- **Probabilistic outputs** — generates diverse outputs each time, not deterministic results

KEY INSIGHT: Generative AI is fundamentally different from traditional AI. While a discriminative model learns $P(\text{label} \mid \text{data})$, a generative model learns $P(\text{data})$ itself — allowing it to sample entirely new examples that look like real data.

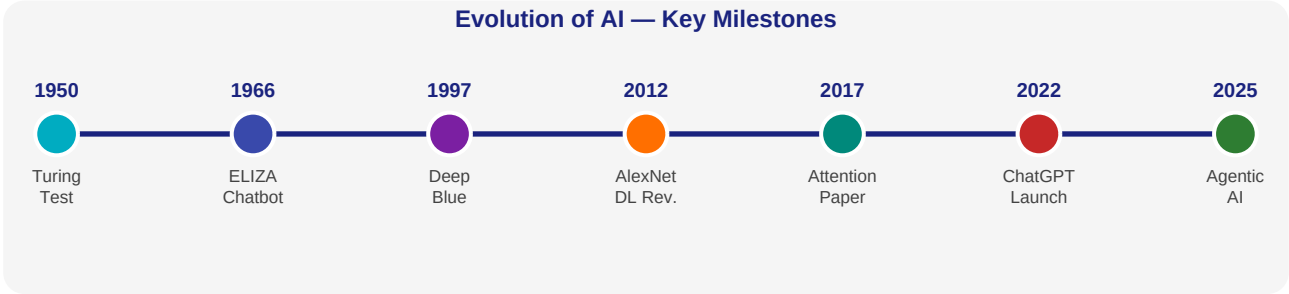


Figure 2: The Evolution of Artificial Intelligence — Key Milestones

2

Generative AI Architectures — Transformers & Beyond

2.1 Major Types of Generative Models

Three major architectural families power modern Generative AI. Each uses a fundamentally different mathematical approach to learn and generate data:

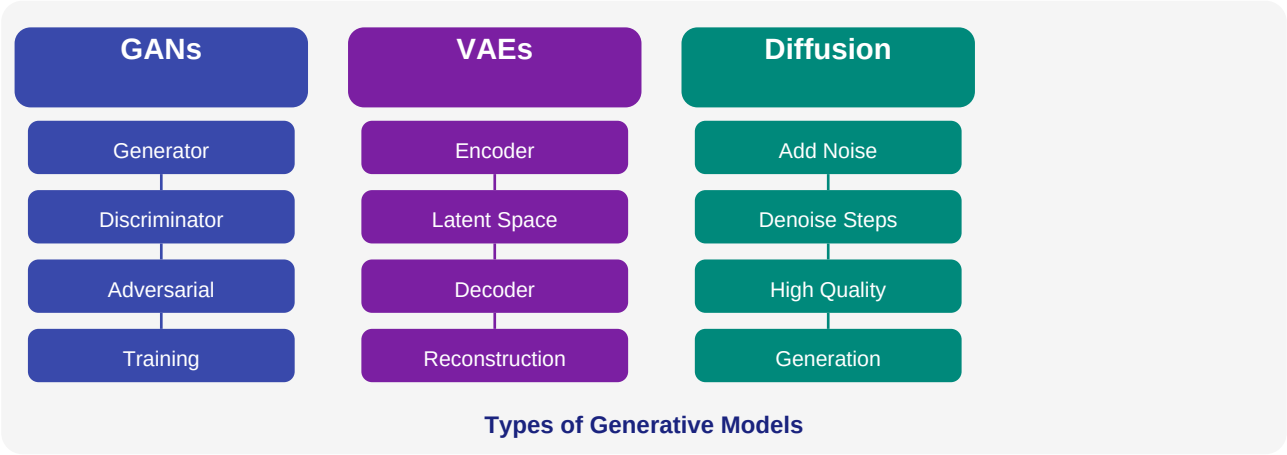


Figure 3: Three Major Generative Model Architectures and Their Pipelines

Model Type	Core Mechanism	Strengths	Key Applications
GANs (Generative Adversarial Networks)	Generator vs. Discriminator competition	Photorealistic image quality	Deepfakes, Image Synthesis, Art
VAEs (Variational Autoencoders)	Encode to latent space, decode to reconstruct	Smooth latent space, stable training	Image editing, Anomaly detection, Molecular design
Diffusion Models	Gradually add then remove noise iteratively	Highest quality image generation	Stable Diffusion, DALL-E 2, Midjourney

2.2 The Transformer — The Backbone of Modern AI

Introduced in the landmark 2017 paper '*Attention Is All You Need*' by Vaswani et al., the Transformer architecture revolutionized NLP and became the foundation for all major LLMs. Its key innovation was replacing sequential processing (RNNs/LSTMs) with a parallel **self-attention mechanism** that allows every token to directly attend to every other token simultaneously.

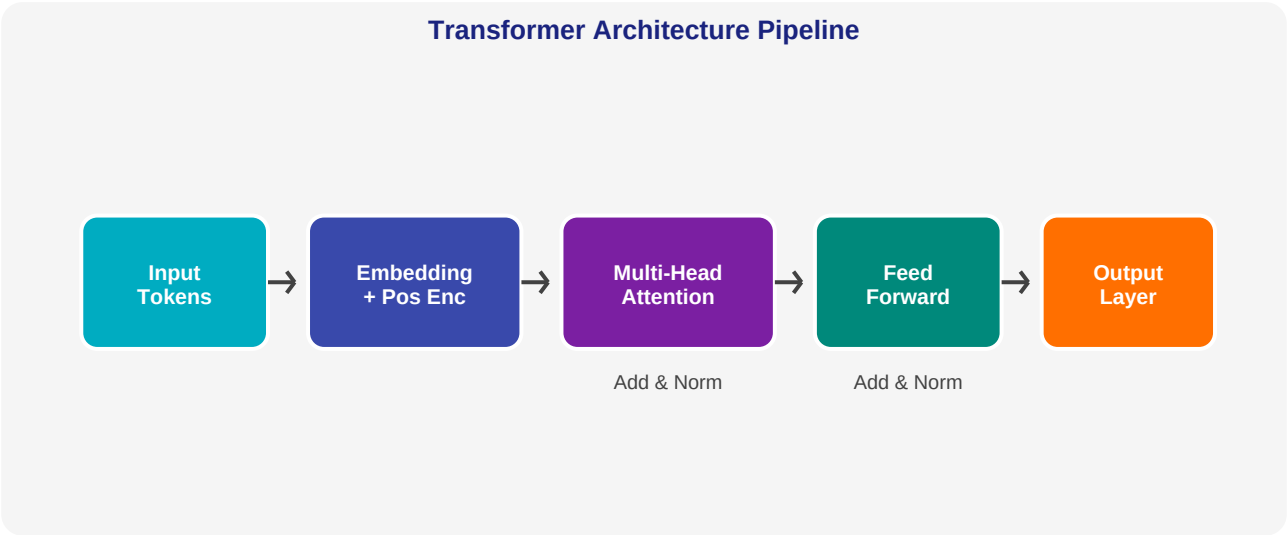


Figure 4: Complete Transformer Architecture Pipeline — From Input Tokens to Output

2.3 Core Components of the Transformer

- Tokenization:** Text is split into sub-word tokens (e.g., 'unbelievable' → ['un', '##believ', '##able']). Each token maps to an integer ID from a fixed vocabulary.
- Embedding Layer:** Each token ID is converted into a dense vector of numbers (e.g., 768 dimensions) capturing its semantic meaning in a high-dimensional space.
- Positional Encoding:** Since attention has no inherent order, sinusoidal position signals are added to embeddings so the model knows token order (word 1 vs word 100).
- Multi-Head Self-Attention:** The model computes Query (Q), Key (K), and Value (V) matrices for each token. Attention scores = $\text{Softmax}(QK^T / \sqrt{d_k}) \times V$. Multiple heads capture different types of relationships.
- Feed-Forward Network:** Two linear layers with a ReLU activation applied identically to each position, allowing the model to process attended representations further.
- Add & Layer Norm:** Residual connections (Add) prevent vanishing gradients; Layer Normalization stabilizes training and speeds convergence across the deep stack.

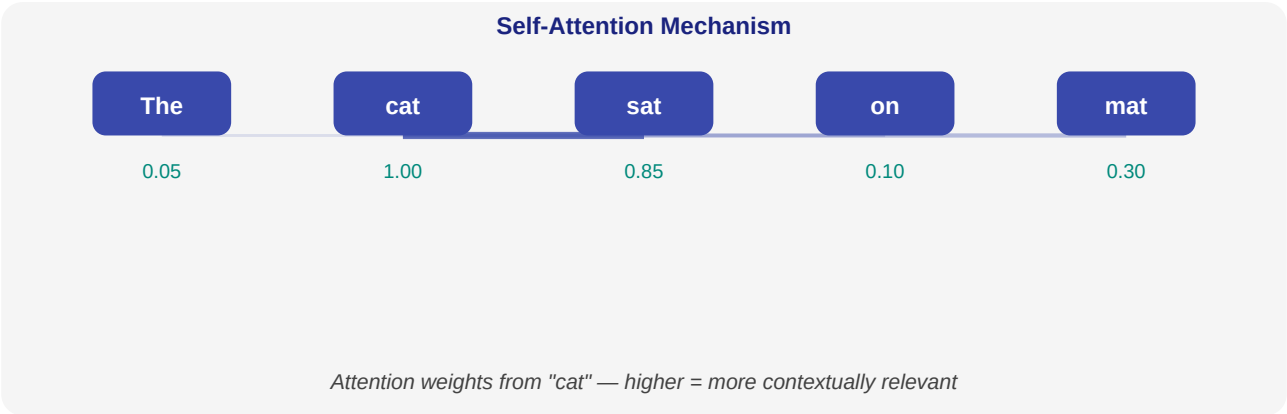


Figure 5: Self-Attention — how 'cat' attends to related words in context

3

Generative AI Architecture & Its Applications

3.1 GPT vs BERT — Two Dominant Paradigms

Feature	GPT (Decoder-Only)	BERT (Encoder-Only)
Architecture	Causal / Autoregressive	Bidirectional / Masked
Pre-training Task	Predict next token	Masked Language Modeling
Direction	Left-to-right only	Full context (both directions)
Best For	Text generation, chatbots	Classification, Q&A
Key Models	GPT-4, Claude, LLaMA	BERT, RoBERTa, ALBERT

3.2 The 2024–2025 AI Tool Landscape

The period 2024–2025 marked a major leap in multimodal AI, with models capable of handling text, images, audio, and video simultaneously. Specialized tools emerged across every domain:

Category	Leading Tools	Key Capability
Text & Reasoning	GPT-4o, Claude 3.5 Sonnet, Gemini 1.5 Pro	Complex reasoning, long context
Image Generation	Midjourney v6, DALL-E 3, Adobe Firefly	Photorealistic image synthesis
Video Generation	Sora (OpenAI), Runway Gen-3, Luma Dream Machine	Text-to-video generation
Code Assistance	GitHub Copilot, Cursor, Replit Ghostwriter	Code generation & debugging
Multimodal	GPT-4o, Gemini Ultra, Claude 3 Opus	Cross-modal understanding
Design	Canva AI, Adobe Firefly, Stable Diffusion	Creative content generation

3.3 Real-World Applications of Generative AI

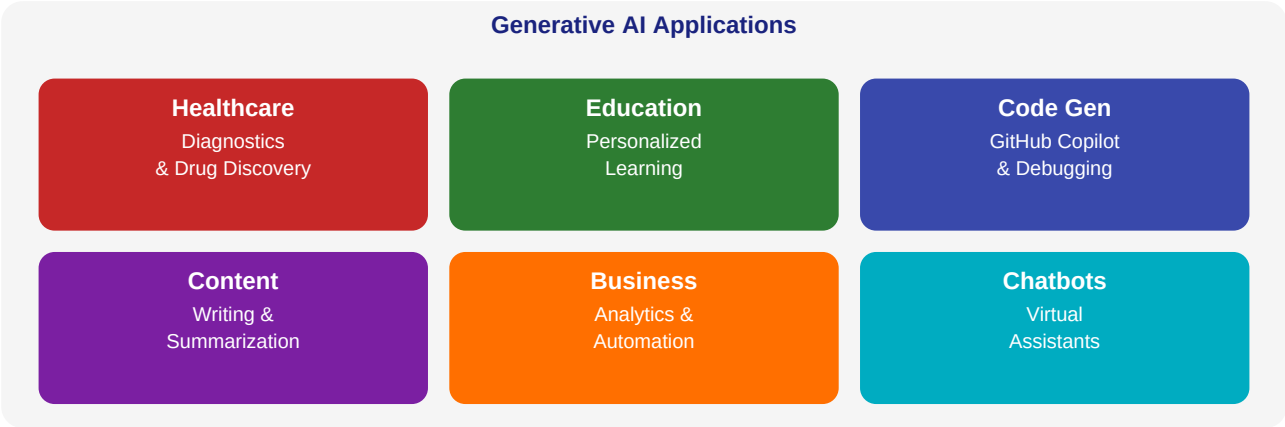


Figure 6: Generative AI Application Domains Across Industries

3.4 Domain Deep Dives

■ Healthcare & Medicine

Generative AI assists in drug discovery by generating novel molecular structures, helps radiologists detect anomalies in medical images, and powers diagnostic chatbots that triage patient symptoms. Models trained on clinical data can generate synthetic patient records for research without privacy risks.

■ Education & Personalized Learning

AI tutors adapt to each student's pace, generating customized explanations, quizzes, and feedback. Platforms like Khan Academy and Duolingo embed LLMs to provide Socratic questioning and instant homework help, democratizing access to high-quality education globally.

■ Software Engineering

Tools like GitHub Copilot complete code from natural language descriptions, reducing boilerplate coding time by up to 55% (GitHub, 2023). LLMs also explain existing code, generate unit tests, detect bugs, and suggest security improvements automatically.

■ Business & Automation

Enterprises use LLMs for automated report generation, customer service chatbots, contract analysis, and market research summarization. Agentic AI systems can now execute multi-step workflows autonomously — browsing the web, calling APIs, and writing emails without human intervention.

4

The Impact of Scaling in Large Language Models

4.1 The Scaling Hypothesis

One of the most important empirical discoveries in AI is the **Scaling Law**: model performance improves predictably as you increase three key variables — the number of model parameters (N), the amount of training data (D), and compute budget (C). This was formalized by Kaplan et al. (2020) at OpenAI and later refined by Hoffmann et al. in the *Chinchilla* paper (2022).

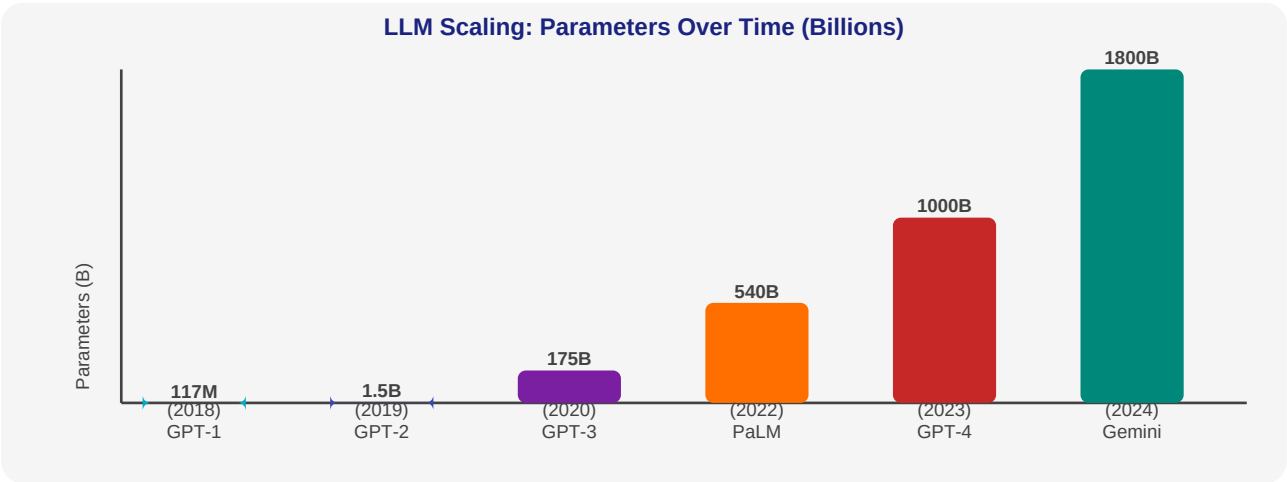


Figure 7: Exponential Growth in LLM Parameter Count (2018–2024)

4.2 What Happens When You Scale?

Scale Dimension	What Improves	Example
More Parameters (Depth & Width)	Richer world knowledge, better reasoning chains	GPT-3 (175B) outperformed GPT-2 on all benchmarks
More Training Data (Tokens)	Better generalization, less overfitting	Chinchilla: 70B model trained on 1.4T tokens beat 280B model
More Compute (GPU/TPU Hours)	Longer training, better convergence	Each GPT generation requires ~10x more compute
Longer Context Windows	Document-level reasoning, better memory	Claude 3.5: 200K tokens; Gemini 1.5: 1M tokens

4.3 Emergent Capabilities — Surprises at Scale

A striking phenomenon is **emergence**: capabilities that appear suddenly and unpredictably once models cross certain scale thresholds, without being explicitly trained for them:

- **Few-Shot Learning**: GPT-3 at 175B parameters could learn new tasks from just 2–3 examples in the prompt, with zero gradient updates
- **Chain-of-Thought Reasoning**: Models above ~100B parameters spontaneously learned to 'think out loud' through multi-step math and logic problems
- **Code Generation**: LLMs learned to write functional programs despite being trained primarily on natural language text

- **Instruction Following:** Large models began following complex, multi-part instructions without explicit fine-tuning
- **Language Translation:** Models trained only on English text could translate between other languages after scaling

CHINCHILLA SCALING LAW (Hoffmann et al., 2022): For a given compute budget, the optimal strategy is to scale model size and training tokens equally. For every doubling of parameters, training data should also double. This overturned the earlier belief that bigger models always win.

4.4 Costs and Challenges of Scaling

Scaling is not without significant challenges. Training GPT-4 was estimated to cost over \$100 million in compute alone. Environmental impact is substantial — a single large model training run can emit as much CO2 as several transatlantic flights. The field is actively researching **efficient scaling** through techniques like:

- Mixture of Experts (MoE) — activate only a subset of parameters per forward pass
- Quantization — reduce precision from 32-bit to 4-bit floats with minimal quality loss
- Knowledge Distillation — train smaller 'student' models to mimic larger 'teacher' models
- Sparse Attention — limit attention to local windows instead of full quadratic attention
- Flash Attention — hardware-aware memory-efficient attention computation

5

Large Language Models — How They Are Built

5.1 What is a Large Language Model?

A Large Language Model (LLM) is a massive neural network — built on the Transformer architecture — trained on trillions of tokens of text to model the probability distribution of language. At its core, an LLM does one thing: **predict the most likely next token** given all previous tokens. Through this simple objective applied at enormous scale, LLMs acquire broad world knowledge, reasoning ability, and the capacity to follow complex instructions.

5.2 The Three-Stage Training Pipeline

Modern LLMs like GPT-4, Claude, and Gemini are built through three carefully orchestrated phases:

Phase 1: Pre-Training	The model processes trillions of tokens (books, Wikipedia, web pages, code repositories) and learns to predict the next token at each position. This builds the model's core knowledge base — grammar, facts, reasoning patterns, and world understanding. Training uses the cross-entropy loss function: $L = -\sum \log P(\text{token}_t \mid \text{token}_1, \dots, \text{token}_{\{t-1\}})$. Pre-training a GPT-4 sized model requires months on thousands of GPUs.
Phase 2: Supervised Fine-Tuning (SFT)	Human contractors create thousands of high-quality (prompt, ideal-response) pairs covering a wide range of tasks. The pre-trained model is fine-tuned on these examples to learn the desired format and behavior — how to be helpful, structured, and instructable rather than just predicting the next internet word.
Phase 3: RLHF (Reinforcement Learning from Human Feedback)	Human raters compare pairs of model responses and label which is better. These preferences train a Reward Model (RM) that predicts response quality. The LLM is then fine-tuned using Proximal Policy Optimization (PPO) to maximize reward — aligning the model with human values: helpfulness, harmlessness, and honesty (the '3H' framework).

5.3 Data Requirements

Volume: Trillions of tokens — GPT-3 used 300B tokens; modern models use 1–15 trillion
Diversity: Books, code, scientific papers, websites, multilingual content for broad generalization
Quality: Aggressive deduplication, toxicity filtering, and quality scoring pipelines
Compute: Thousands of A100/H100 GPUs, specialized interconnects, petabytes of storage

5.4 Technical Implementation — Self-Attention in Code

The core mathematical operation of a Transformer is the scaled dot-product attention:

```
import numpy as np

def scaled_dot_product_attention(query, key, value, mask=None):
```

```
"""
Core Transformer Self-Attention Mechanism

Args:
    query: Q matrix [seq_len, d_k]
    key:    K matrix [seq_len, d_k]
    value: V matrix [seq_len, d_v]
Returns:
    output: attended representation
    weights: attention distribution (interpretable)
"""

d_k = query.shape[-1] # Key dimension for scaling

# Step 1: Compute raw attention scores (similarity)
scores = np.dot(query, key.T) / np.sqrt(d_k)

# Step 2: Apply optional causal mask (for GPT-style models)
if mask is not None:
    scores = scores + (mask * -1e9) # -inf for masked positions

# Step 3: Softmax – convert scores to probability distribution
exp_scores = np.exp(scores - np.max(scores)) # numerically stable
weights = exp_scores / np.sum(exp_scores, axis=-1, keepdims=True)

# Step 4: Weighted sum of values
output = np.dot(weights, value)

return output, weights

# Example: "cat" in "The cat sat on the mat"
d_k = 4 # embedding dimension (tiny for illustration)
Q = np.random.randn(1, d_k) # Query: "cat" asking what to attend to
K = np.random.randn(6, d_k) # Keys: all 6 tokens in sentence
V = np.random.randn(6, d_k) # Values: semantic content of each token

context_vector, attention_weights = scaled_dot_product_attention(Q, K, V)
print(f"Attention weights: {attention_weights}") # which tokens "cat" attends to
print(f"Context vector: {context_vector}") # enriched representation of "cat"
```

5.5 Limitations and Ethical Considerations

Limitation / Issue	Description	Mitigation Approach
Hallucinations	Generates confident but factually wrong information	RLHF, RAG, fact-checking layers
Data Bias	Inherits biases from training corpus	Curated datasets, bias audits
High Compute Cost	Training costs millions of dollars	Efficient architectures (MoE, QLoRA)
Privacy Risks	May memorize and leak training data	Differential privacy, data filtering
Misinformation	Can generate convincing fake content	Watermarking, detection tools
Job Displacement	Automates knowledge worker tasks	Reskilling programs, regulation

5.6 Future Trends

- **Multimodal 'Omni' Models:** Systems like GPT-4o process text, images, audio, and video in a single unified model
- **Agentic AI:** Models that plan, execute tools, browse the web, and complete multi-step tasks autonomously
- **Smaller & More Efficient Models:** Techniques like distillation and quantization bring frontier capability to edge devices
- **Better Alignment & Safety:** Constitutional AI, interpretability research, and robustness testing
- **Personalized AI:** Models that adapt to individual user style, knowledge, and long-term memory

References

- [1] Vaswani, A., et al. (2017). Attention Is All You Need. NeurIPS.
- [2] Kaplan, J., et al. (2020). Scaling Laws for Neural Language Models. arXiv.
- [3] Hoffmann, J., et al. (2022). Training Compute-Optimal Large Language Models (Chinchilla). arXiv.
- [4] OpenAI (2024). GPT-4o Technical Report.
- [5] Goodfellow, I., et al. (2014). Generative Adversarial Networks. NeurIPS.
- [6] Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep Learning. MIT Press.
- [7] Google AI Research. (2024). Gemini: A Family of Highly Capable Multimodal Models.